

025 实验 多层前向与XOR

这份实验独立于正文，可在普通CPU上完成。目标是用可追踪的前向计算确认隐藏表示改变了什么，并用相同数据区分同权重消融、有限候选搜索和未见点预测。先修024；不需要自动微分、GPU或网络账户。先完成A与B的纸笔部分，再运行脚本，最后打开详解。

环境与运行顺序

将本单元完整目录放在同一位置，不要只下载Notebook。建议从终端进入该目录：

```
conda env create -f environment.yml
conda activate dl-unit-025
python experiment.py
python test_experiment.py
jupyter lab
```

在Jupyter中打开experiment.ipynb，重启内核，再从上到下运行全部单元。核心脚本与测试只需要Python、NumPy；生成图另外需要matplotlib。随附环境文件是安装建议，制作环境并未实际新建Anaconda环境，也没有实际测试浏览器Jupyter界面或跨平台安装。随附Notebook已在新的Python进程中用真实IPython InProcessKernel顺序执行；这不等于验证了外部内核的socket通信。

默认脚本覆盖outputs/中的4个同名结果。想保留另一份复跑，可用python experiment.py --output my_run。默认输入路径相对脚本文件本身，切换当前工作目录不改变输入。Notebook写notebook_outputs/并逐字节与发布结果比较；这个重跑目录无需提交。

A 先算网络再看概率

四条输入依次为(0,0), (0,1), (1,0), (1,1)，标签为(0,1,1,0)。采用以下参数：

$$W_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad b_1 = (0, -1), \quad W_2 = \begin{bmatrix} 2 \\ -4 \end{bmatrix}, \quad b_2 = -1.$$

1. 写出 $X, W_1, b_1, Z_1, H_1, W_2, b_2, Z_2$ 的代码形状。代码中的偏置是一维数组，其他网络中间量均保留二维。
2. 对每行依次计算 $Z_1 = XW_1 + b_1$ 、 $H_1 = \max(0, Z_1)$ 、 $Z_2 = H_1W_2 + b_2$ 。先不算sigmoid，把结果记录成4行。
3. 将 $Z_2[:, 0]$ 变成一维logit，再算概率。为什么这一步使用 $[:, 0]$ 而不是随意reshape？如果输出实际有两列，会发生什么？
4. 用logit大于或等于0预测1。计算正确率及平均交叉熵。解释“全判对”为什么不等于“损失为0”。
5. 追加输入(0.25,0.5)，给出每个中间值。不要给它编造真实标签。

先写出你自己的数字，再查看outputs/forward.csv。该表中a1,a2表示激活前值，h1,h2为激活后值；logit与probability来自有ReLU的尺度1构造。每一列的意义见数据字典。

B 精确改变一个机制

把唯一隐藏激活换成恒等函数，所有权重和偏置保持不变。恒等函数就是输入是什么，输出还是什么。使用核心API：

```
import numpy as np
import experiment as e
X, y, cfg, ids = e.load_inputs()
logits_relu, trace = e.forward(X, e.xor_layers())
logits_affine, _ = e.forward(
    X, e.xor_layers(), activation="identity"
)
W, b = e.collapse_affine(e.xor_layers())
np.testing.assert_allclose(logits_affine, X @ W + b)
```

自己手算合并后的 W, b ，然后逐行核对程序。回答：哪个点改变了类别？这个实验是否证明“经过充分训练的所有线性模型损失都比0.997093大”？要拒绝这个说法，你只需构造一个更好的仿射候选。

在纸上重新证明：不论如何选择单个仿射logit的两个权重和一个偏置，四条XOR真值不能全判对。测试某一组权重的失败，只能否定这组权重；四个分类不等式的矛盾才能否定整个仿射函数族。

C 完整检查130个候选

脚本已经规定两个互不混淆的搜索空间：仿射参数三个坐标各取 $-2, -1, 0, 1, 2$ ，共125组；ReLU构造仅把输出权重和输出偏置同时乘以 $\gamma = 0, 1, 2, 4, 8$ ，共5组。搜索使用全部四个真值表点，不保留所谓测试点。

读取outputs/search.csv，逐项回答：

1. 为什么候选数为125而不是15？为什么第二个搜索只有5个候选而不是 5^9 ？
2. 找出仿射最小交叉熵的参数、损失及正确率；再找仿射最高正确率。两者是否必须由同一候选达到？
3. 对所有尺度，使用公式 $L(\gamma) = \log(1 + e^{-\gamma})$ 独立复算损失。尺度0处采用什么平局规则？
4. 如果把尺度上界提高到16，能否仍把8称为这个新集合的最佳尺度？这个变化会改变4点正确率吗？
5. 这两个搜索是否公平比较了两个架构在相同训练算法下的计算效率？说明它们各自能支持什么结论。

脚本的输入参数`--config`与`--samples`可以指向副本，便于验证输入契约，但本讲固定教学运行只接受默认配置和原始真值表。需要探索尺度16时，直接调用`e.xor_layers(16)`与`e.binary_metrics`并在你自己的记录里报告；不要修改默认配置后仍展示固定教材图。核心前向API与固定教材运行入口的用途不同。

D 在图上区分三种边界与两种延伸

正文图3显示从输入坐标到隐藏坐标的映射，图4显示分类区域，图5显示沿 $s = x_1 + x_2$ 的logit。请分别找到：

- ReLU状态切换的位置
- logit等于0的位置
- 概率等于0.5的位置

后两者在本讲同义，第一项不同。推导 $[0, 1]^2$ 内分类区域的两个不等式，并核对四角点。

再独立实现 $z_B = 2|x_1 - x_2| - 1$ 。先用两个ReLU写出绝对值，再构造它的两层参数，不要调用现成训练器。将它与主网络在四角点、中心(0.5, 0.5)和(0.25, 0.5)比较。允许比较数值和预测；除四角点外，当前没有真实标签，不能比较准确率。

E 深度复合与形状失败

给定 $T(t) = 2 \operatorname{ReLU}(t) - 4 \operatorname{ReLU}(t - 1/2)$ ，只考察 $t \in [0, 1]$ 。手算输入1/8经过1、2、3次复合的结果，再用`e.triangle([0.125], depth=3)`核对。读取`outputs/depth.csv`，说明17个网格点为什么只是图形采样，不是“证明所有连续输入上函数正确”的证据。完整分段公式才是证明的依据。

有意制造以下错误，各自先预测后果，再运行：

```
# 这些调用应该报错；不要删除验证来让它们通过。
e.forward([[0, 1]], [(np.ones((2, 2)), np.zeros((1, 2)))]))
e.forward([[True, 0]], e.xor_layers())
e.binary_metrics([0, 1], [[0], [1]])
```

错误分别涉及偏置维度、把布尔值当实数坐标、标签维度。再观察为什么`np.max(Z1, axis=0)`返回(2,)而不是(4,2)：它把样本轴归约掉了，不是逐元素ReLU。

F 验证不只看正常输入

运行测试后，阅读而不是跳过`test_experiment.py`中以下独立参照：

- 用Python Fraction按分量计算180个小网络，分别核对ReLU与恒等前向；再核对仿射合并结果与参数计数
- 用60位Decimal的指数和对数重算全部130个候选，不调用NumPy损失函数作为“另一份答案”

- 用反射公式 $2 \min(t, 1 - t)$ 核对606个复合值
- 30种API输入拒绝、两种中间溢出、14种CSV/JSON问题，以及序列化失败和路径符号链接检查

失败测试同时检查新输出目录不存在、已有结果逐字节保持原状。这样能发现“先写一半，再因后面输入不合法而失败”的错误。本单元未实现磁盘故障、断电、并发进程路径交换下的跨文件事务；不要将输入失败保护解释成任意故障都不会破坏文件。

还可以在临时目录里亲手验证“先失败、后不写”的性质。下面只改一条标签，固定教学入口必须拒绝；临时目录会自动清理，不动随附数据与结果。

```
from pathlib import Path
from tempfile import TemporaryDirectory
with TemporaryDirectory() as temporary:
    folder = Path(temporary)
    changed = folder / "changed.csv"
    changed.write_text(
        e.SAMPLE_TEXT.replace("b,0,1,1", "b,0,1,0"),
        encoding="utf-8",
    )
    destination = folder / "should_not_exist"
    try:
        e.run(destination, samples=changed)
    except ValueError:
        assert not destination.exists()
    else:
        raise AssertionError("changed teaching data were accepted")
```

这个失败不是说改变标签永远不允许，而是说固定教材结果的解释不适用于这个新问题。若研究另一个任务，应使用独立数据、计算与说明。再阅读测试中的已有输出保护断言，解释为什么只检查“程序报错”不足以发现旧文件被提前覆盖。

应提交的自学记录

保存A的完整手算、B的仿射不可能性证明、C的两种搜索结论、D的未见点对照、E的三个报错解释，以及脚本/Notebook四份结果一致的检查记录。最后用一段话分别回答：已经证明能表示什么、实际执行了什么参数选择、还有什么泛化问题没有证据。完整参考答案见单独的answers.pdf，但先保留自己的推理。